Module - 2.

1. ~~logic~~ Problem :- Fraud Transaction Detection

logic

→ Compare the suspicious transaction id with each transiction ID one by one.

→ If a match is found, stop searching and print "Fraud transaction Found".

→ If no match is found after checking all IDs print "Transaction Not found".

Approach

→ Read the number of transiction Ids
→ Store all transiction IDs in an array
→ Input the suspicious transaction ID.
→ Traverse the array using linear search
→ If id is found print "Fraud transaction" otherwise print "transaction not found".

Dry run

N=5

Tran IDs = 1201 1305 1450 1408 1502

Suspicious IDs = 1450

Iteration	Curren ID	Compare with 1450	Result
1	1201	No	continue
2	1305	No	continue
3	1450	Yes	Found → stop

2. Problem → Server log search

Logic:

→ Compare the Target log ID with the middle element of the Sorted array.
→ If the middle element matches, the log is found.
→ If the target is greater, search the right half otherwise, search the left half.
→ Repeat until the log is found or search range becomes empty.

Approach

→ Read the number of log IDs
→ Store the Sorted log IDs in an array
→ Initialize start and end
→ Find the middle element and compare it with the target.
→ If found print "log found" else print "log not found".

Dry Run

N = 7
log IDs = 1001 1005 1010 1015 1020 1030 1040
Target = 1015

Step	Start	End	mid	value	result
1	0	6	3	1015	match found

Date:
Page:

3. Problem :— Sort Employee Salaries

Logic
→ Compare adjacent employee salaries
→ If the left salary is greater than the right salary swap them
→ Repeat this process until all salaries are arranged in ascending Order.

Approach
→ Read the number of employes
→ Sort all the salaries in an array
→ Use bubble sort to compare and swap adjacent elements
→ Repeat for all passes until the array is sorted
→ Print the sorted salaries.

Dry run
N = 5

45 000 32000 55000 28 000 60000

Pass	Array				
Initial	45000	32 000	55000	28K	60K
Pass 1	32000	45 000	28000	55 K	60K
Pass 2	32000	28 000	45000	55 K	60K
Pass 3	28000	32 000	45000	55K	60K
Pass 4	28000	32 000	45000	50 K	60K

Date:
Page:

4 Problem :- Sort website Response time

Logic

→ Take one element at a time from unsorted part
→ Compare it with previous elements and shift larger elements to right.
→ Insert the element at its correct position.
→ Repeat until the array is sorted.

Approach

→ Read the number of response time
→ Store all the response time in an array
→ Start from the second element and treat it as the key
→ Shift larger elements to the right and insert the key at the correct position.
→ Print the sorted response time.

Dry run

n = 5

750 120 320 180 100

Pass	Array				
Initial	750	120	320	180	100
Pass 1	120	750	320	180	100
Pass 2	120	750	320	180	100
Pass 3	120	180	750	320	100
Pass 4	100	120	180	250	320

Date:
Page:

5 Problem :- Prioritize bug reports

logic

→ Find the smallest priority score in the unsorted part of array.

→ Swap it with the first unsorted element.

→ Repeat the process until all priority scores are sorted.

Approch

→ Read the number of bug reports

→ Stores all priority scores in an array

→ Find the minimum element in the unsorted part

→ swap it with the current position

Dry run

N=5

9 2 8 1 5

Pass	Array
initial	~~12~~ 92815
Pass 1	12 8 95
Pass 2	1285
Pass 3	1258
pass 4	1258 9.